COLLÈGE LA CITÉ · SCIENCE DES DONNÉES APPLIQUÉES

Intelligence artificielle appliquée au prétraitement des données

Évaluation sommative 2 (UA2) · Document technique

Pipeline de prédiction des annulations hôtelières

Du jeu Hotel Booking Demand à la décision : pipeline scikit-learn complet (prétraitement, transformateurs personnalisés, sélection des caractéristiques, modèle calibré), analyse prescriptive des pertes et des acomptes, application Streamlit.

Remis par

Yacine Lezoul · 2748526

Professeure	Nadjate Saïdani
Cours	Intelligence artificielle appliquée au prétraitement des données (030513 IFM)
Date de remise	11 novembre 2026
Livrables	Notebook 01_pipeline.ipynb, diapositives, ce document
Dépôt GitHub	https://github.com/lezoul/hotel-cancellation-pipeline

Sommaire

01 Vue d'ensemble
02 Architecture du projet
03 Données et préparation
04 Pipelines de prétraitement
05 Transformateurs personnalisés
06 Sélection des caractéristiques
07 Modélisation et évaluation
08 Analyse prescriptive
09 Application Streamlit
10 Reproductibilité
11 Limites et pistes d'amélioration

Démarche suivie

Exploration → Nettoyage (hors pipeline) → Pipelines de prétraitement → Transformateurs personnalisés → Sélection des caractéristiques → Comparaison des modèles (F2) → Optimisation → Calibration → Seuils → Évaluation sur le test → Analyse prescriptive → Application Streamlit

SECTION 01

Vue d'ensemble

En bref Le modèle retenu (HistGradientBoosting pondéré et calibré) détecte 93 % des annulations au seuil F2 et atteint 75 % d'alertes justes au seuil F0,5, sur 17 197 réservations de test jamais vues. Les annulations sans dépôt coûtent 3,0 M€ net à l'hôtel sur la période.

Problème. Une entreprise hôtelière veut prédire le risque d'annulation de chaque réservation (`is_canceled`) à partir des informations connues au moment de la réservation. Il s'agit d'une classification binaire : 27 variables explicatives, 85 982 réservations après nettoyage.

1.1 Livrables et où ils se trouvent

Livrable	Emplacement
Pipeline scikit-learn complet (prétraitement, transformateurs, sélection, modèle)	`src/pipelines.py`, `src/transformers.py`
Exécution et interprétation des Parties 1 à 5 de la consigne	`notebooks/01_pipeline.ipynb`
Analyse prescriptive : coût des annulations, revente, acomptes, surréservation	`notebooks/02_analyse_prescriptive.ipynb`
Application interactive	`app/app.py` (Streamlit)
Diapositives de présentation	`reports/presentation_annulations.pptx`
Document technique	`docs/document_technique.docx` (ce document)

1.2 Résultats clés

Indicateur	Valeur
ROC-AUC (test)	0,895
F2 maximal (test / validation croisée)	0,798 / 0,792
Rappel / précision au seuil F2 (0,17)	93 % / 51 %
Rappel / précision au seuil F0,5 (0,54)	59 % / 75 %
Score de Brier (calibré / sans modèle)	0,115 / 0,200
Manque à gagner brut / net des annulations sans dépôt	13,1 M€ / 3,0 M€

SECTION 02

Architecture du projet

En bref Le code réutilisable est dans `src/`, importé par les deux notebooks et par l'application ; les notebooks exécutent, affichent et interprètent. Un seul pipeline entraîné sert partout, sans duplication du prétraitement.

Arborescence commentée

```
hotel-cancellation-pipeline/
├── data/
│   ├── raw/hotels.csv            # données brutes (téléchargées automatiquement)
│   └── README.md                 # source et description des données
├── src/
│   ├── config.py                 # chemins, colonnes de fuite, graine aléatoire
│   ├── data_loader.py            # chargement, diagnostic, nettoyage, séparation 80/20
│   ├── transformers.py           # 3 transformateurs personnalisés (TransformerMixin)
│   ├── pipelines.py              # tous les pipelines, sur le modèle du cours
│   ├── evaluation.py             # métriques, scorer F2, graphiques, bilan de sélection
│   ├── prescriptive.py           # délais, revente, pertes, acomptes, surréservation
│   └── sauvegarde.py             # enregistrement et chargement du pipeline
├── notebooks/
│   ├── 01_pipeline.ipynb         # Parties 1 à 5 de la consigne
│   └── 02_analyse_prescriptive.ipynb
├── app/app.py                    # application Streamlit
├── models/                       # pipeline entraîné, métadonnées, ensemble de test
├── reports/
│   ├── figures/                  # graphiques exportés par les notebooks
│   └── presentation_annulations.pptx
├── docs/document_technique.docx  # ce document
├── tests/test_transformers.py    # 12 tests unitaires
├── requirements.txt              # versions fixées
└── README.md
```

Flux d'exécution. `01_pipeline.ipynb` entraîne le pipeline et l'enregistre dans `models/`. `02_analyse_prescriptive.ipynb` et `app/app.py` rechargent ce même objet : le prétraitement n'est jamais réécrit.

SECTION 03

Données et préparation

En bref Le jeu brut compte 119 390 réservations et 32 colonnes. Après nettoyage, il reste 85 982 réservations, 27 variables explicatives et 27,6 % d'annulations. Trois colonnes de fuite de données sont exclues du modèle.

Source. Hotel Booking Demand (Antonio, Almeida et Nunes, 2019), version TidyTuesday du 11 février 2020 : deux hôtels portugais, arrivées de juillet 2015 à août 2017. Le fichier est téléchargé automatiquement par `load_raw_data()` s'il est absent.

3.1 Problèmes de qualité détectés

Problème	Ampleur	Traitement
Lignes strictement identiques	31 994	supprimées avant la séparation
Réservations sans aucun client	180	supprimées
Tarif négatif ou aberrant (5 400 €)	2	supprimées
`company` / `agent` manquants	94 % / 14 %	indicateurs `has_company`, `has_agent`
`country` / `children` manquants	488 / 4	imputation dans le pipeline
Modalités « Undefined »	1 176	corrigées dans le pipeline
Valeurs extrêmes (`lead_time` jusqu'à 737 jours)	au-delà des 1er et 99e centiles	écrêtage dans le pipeline
177 pays, dont 157 sous 0,5 % des réservations		regroupement dans le pipeline

3.2 Fuite de données

`reservation_status` reproduit exactement la cible (Canceled et No-Show = 1, Check-Out = 0). `reservation_status_date` et `assigned_room_type` ne sont connues qu'après l'issue de la réservation : une chambre attribuée différente de la chambre réservée s'accompagne de 5,4 % d'annulations contre 41,6 % sinon. Ces trois colonnes sont exclues du modèle. `arrival_date_year` est aussi exclue, car une année future n'existe pas dans l'apprentissage.

3.3 Répartition du nettoyage

Règle suivie. Tout ce qui doit s'appliquer à une nouvelle réservation est fait dans le pipeline. Ce qui supprime des lignes de l'historique est fait une seule fois, avant la séparation, par `clean_data()`, pour deux raisons :

1. Dans un `Pipeline`, `transform()` reçoit seulement `X` : supprimer des lignes désalignerait `X` et `y`.
2. Les doublons doivent être retirés avant `train_test_split`, sinon une même réservation se retrouve dans l'entraînement et le test.

Après retrait des colonnes de fuite, 33 241 doublons sont supprimés : des lignes qui ne différaient que par la date de statut deviennent identiques.

Séparation. 80 % / 20 % stratifiée sur la cible, `random_state = 42` : 68 785 réservations d'entraînement, 17 197 de test, 27,6 % d'annulations dans les deux. Le test n'est utilisé qu'une fois, pour l'évaluation finale.

SECTION 04

Pipelines de prétraitement

En bref Tous les pipelines sont écrits dans `src/pipelines.py` sur le modèle du cours : un sous-pipeline par type de colonnes, réunis par un `ColumnTransformer`, puis un pipeline complet qui enchaîne prétraitement, sélection et modèle. 27 colonnes brutes deviennent 93 colonnes numériques, sans aucune valeur manquante.

Structure du pipeline complet

```
model_pipeline = Pipeline([
  ("preprocessing", pipeline_pretraitement)
  │   ├── creation_variables : HotelFeatureEngineer
  │   └── colonnes : preprocessor (ColumnTransformer)
  │         ├── num_continu : SimpleImputer(median) → OutlierCapper → StandardScaler
  │         ├── num_discret : SimpleImputer(median) → StandardScaler
  │         ├── binaire     : SimpleImputer(most_frequent)
  │         └── cat         : SimpleImputer("Inconnu") → RareCategoryGrouper → OneHotEncoder
  ├── ("selection", SelectFromModel(RandomForestClassifier, threshold="median"))
  └── ("model", HistGradientBoostingClassifier)
])
```

4.1 Traitements par groupe de colonnes

Sous-pipeline	Colonnes	Étapes	Justification
`pipeline_continu`	`lead_time`, `adr`, nuits, `total_nights`, `adr_per_person` (6)	médiane, écrêtage 1 % / 99 %, standardisation	distributions très asymétriques ; la médiane résiste aux extrêmes ; l'écrêtage évite qu'ils dominent moyenne et écart-type
`pipeline_discret`	comptages et calendrier (12)	médiane, standardisation	pas d'écrêtage : le 99e centile vaut souvent 0 ou 1 et effacerait des profils rares mais informatifs
`pipeline_binaire`	`is_repeated_guest`, `has_children`, `has_agent`, `has_company` (4)	valeur la plus fréquente	déjà en 0/1
`pipeline_cat`	hôtel, mois, saison, repas, pays, segment, canal, chambre, dépôt, client (10)	constante « Inconnu », regroupement des rares, one-hot	aucun ordre entre modalités ; `handle_unknown="ignore"` protège la prédiction

4.2 Code du ColumnTransformer

Extrait de src/pipelines.py

```
preprocessor = ColumnTransformer([
    ("num_continu", pipeline_continu, colonnes_continues),
    ("num_discret", pipeline_discret, colonnes_discretes),
    ("binaire", pipeline_binaire, colonnes_binaires),
    ("cat", pipeline_cat, colonnes_cat),
], remainder="drop")

pipeline_pretraitement = Pipeline([
    ("creation_variables", HotelFeatureEngineer()),
    ("colonnes", preprocessor),
])
```

Résultat. `pipeline_pretraitement.fit_transform(X_train)` produit 93 colonnes numériques (6 continues, 12 discrètes, 4 binaires, 71 indicatrices), sans aucune valeur manquante. Le test est seulement transformé (`transform`), avec les paramètres appris sur l'entraînement.

Sortie pandas. `set_config(transform_output="pandas")` fait renvoyer un DataFrame à chaque étape, avec des noms de colonnes préfixés par la branche (`num_continu__lead_time`, `cat__country_PRT`). Ces noms permettent de savoir quelles variables la sélection garde.

SECTION 05

Transformateurs personnalisés

En bref Trois classes (`src/transformers.py`) suivent le modèle `NormaliseurMinMax` du cours : `__init__`, `fit` et `transform` sont écrits à la main ; le reste est fourni par scikit-learn.

Classe	Problème traité	fit() apprend	transform() applique	Place dans le pipeline
`HotelFeatureEngineer`	information dispersée, identifiants sans ordre, « Undefined »	rien (règle fixe, comme `SalaireAnnuel`)	crée 7 variables (`total_nights`, `total_guests`, `adr_per_person`, `has_children`, `has_agent`, `has_company`, `arrival_season`) ; retire `agent` et `company`	première étape de `pipeline_pretraitement`
`RareCategoryGrouper`	177 pays, dont 157 sous 0,5 %	les modalités ≥ 0,5 % de chaque colonne	toute autre modalité, y compris inconnue, devient « Autre »	`pipeline_cat`, avant `OneHotEncoder`
`OutlierCapper`	valeurs extrêmes (737 jours de délai)	1er et 99e centiles (`min_`, `max_`)	ramène chaque valeur dans ses bornes	`pipeline_continu`, avant `StandardScaler`

5.1 Écrit à la main ou fourni par scikit-learn

Écrit dans le projet	Fourni par scikit-learn
`__init__` : mémorise les paramètres, sans calcul	`TransformerMixin` : `fit_transform()`
`fit()` : apprend sur l'entraînement, retourne `self`	`BaseEstimator` : `get_params()`, `set_params()`, nécessaires à `clone()` et à la validation croisée
`transform()` : copie X, applique la règle, retourne la copie	`OneToOneFeatureMixin` : `get_feature_names_out()` pour les classes qui gardent les mêmes colonnes
`get_feature_names_out()` de `HotelFeatureEngineer` seulement	`validate_data()` : enregistre `feature_names_in_` et `n_features_in_`

`HotelFeatureEngineer` est la seule exception : il crée des colonnes dont scikit-learn ne peut pas deviner les noms, d'où un `get_feature_names_out()` écrit à la main.

Effet mesuré dans le pipeline entraîné. `country` passe de 168 modalités (entraînement) à 21. `lead_time` est borné à 343 jours, `adr` à 263 €, la durée totale à 14 nuits. Ces paramètres se lisent dans le pipeline entraîné par `named_steps`.

SECTION 06

Sélection des caractéristiques

En bref `SelectFromModel` garde 47 colonnes sur 93 sans dégrader le modèle : F2 maximal de 0,7885 avec sélection contre 0,7889 sans, un écart inférieur à la variation entre plis.

Méthode. Une forêt aléatoire légère (100 arbres, profondeur 12, `class_weight="balanced"`) mesure l'importance de Gini de chaque colonne prétraitée. Les colonnes au-dessus de l'importance médiane (0,0016) sont gardées. Le sélecteur est l'étape `selection` de `model_pipeline` : il est réappris sur chaque pli de validation croisée.

6.1 Justification

Caractéristique des données	Conséquence sur le choix
Colonnes de natures mélangées (continues, binaires, indicatrices)	l'importance d'une forêt se calcule de la même façon pour tous les types ; le chi2 exige des valeurs positives, l'ANOVA suppose une relation linéaire
Relations non linéaires et interactions	une méthode intégrée à un modèle non linéaire les capte
68 785 lignes d'entraînement	RFE réentraînerait le modèle des dizaines de fois ; `SelectFromModel` n'entraîne qu'une forêt
Risque de fuite en validation croisée	intégrée au pipeline, la sélection ne voit jamais le pli de validation

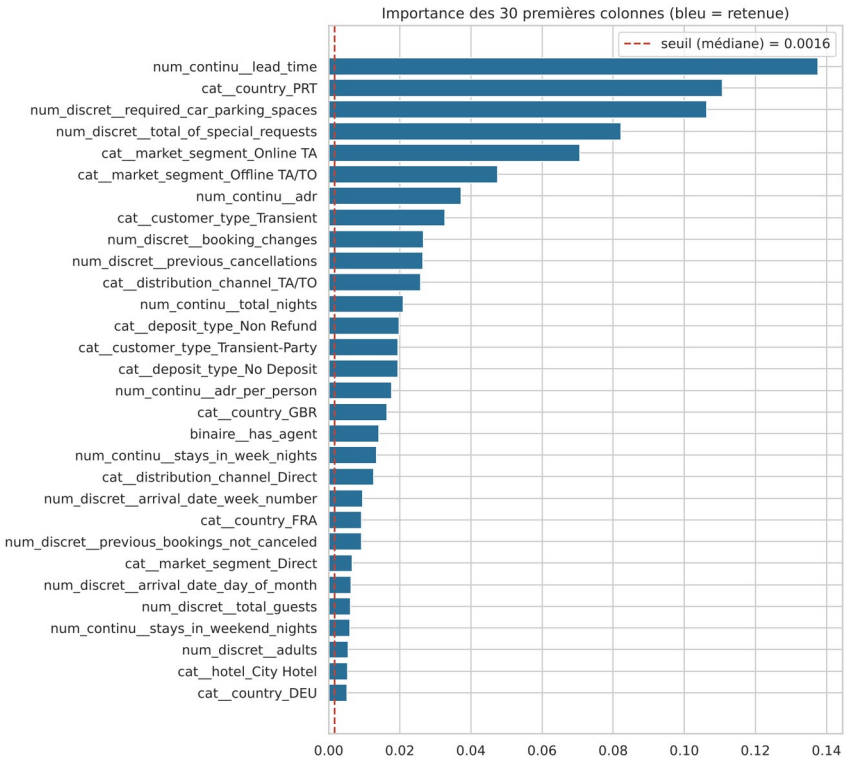


Figure 1. Importance des 30 premières colonnes (bleu : retenue), seuil médian en pointillé

Retenues : `lead_time`, `country`, `market_segment`, `required_car_parking_spaces`, `total_of_special_requests`, `customer_type`, `deposit_type`, `adr`, `booking_changes`, `previous_cancellations` et plusieurs variables créées (`total_nights`, `adr_per_person`, `has_agent`, `has_company`, `total_guests`). **Éliminées :** surtout les 12 indicatrices de mois (l'information est portée par la semaine et la saison), la plupart des types de chambre, les pays rares et certaines formules de repas.

Contrôle. L'information mutuelle, méthode indépendante du modèle, partage 11 des 15 premières colonnes avec la forêt (corrélation de rang de Spearman : 0,58).

SECTION 07

Modélisation et évaluation

En bref Le modèle final est un HistGradientBoosting avec `class_weight="balanced"`, calibré, qui enveloppe le pipeline complet avec un seuil de décision F2 de 0,17. ROC-AUC de 0,895 et 93 % des annulations détectées sur le test.

7.1 Métrique : le F2

Le F2 donne deux fois plus de poids au rappel qu'à la précision : détecter les annulations prime, sans pour autant signaler tout le monde. Optimiser directement le rappel est écarté : prédire « annulée » partout donne un rappel de 100 %.

Formule

$$F_\beta = (1 + \beta^2) \times P \times R / (\beta^2 \times P + R), \quad \text{avec } \beta = 2$$

Le F2 dépend du seuil. Les modèles sont donc comparés sur le **F2 maximal**, calculé au meilleur seuil de chacun (`best_f2_scorer`, `src/evaluation.py`) : sinon un modèle sans pondération serait pénalisé par le seuil arbitraire de 0,5.

Découpage de l'entraînement. Le test (20 %) ne sert qu'à l'évaluation finale. Toutes les décisions sont prises par validation croisée stratifiée dans l'entraînement : 5 plis pour la comparaison des modèles et les seuils, 3 plis pour la recherche d'hyperparamètres.

7.2 Comparaison des modèles

Modèle (même model_pipeline)	F2 maximal	PR-AUC	ROC-AUC	Rappel au seuil 0,5
Forêt aléatoire, balanced	0,792	0,757	0,891	77 %
Forêt aléatoire	0,790	0,761	0,890	58 %
HistGradientBoosting, balanced	0,789	0,751	0,886	82 %
HistGradientBoosting	0,788	0,753	0,886	60 %
Régression logistique	0,739	0,653	0,828	44 %

Les modèles à base d'arbres sont à égalité (écarts de 0,001 à 0,003, l'ordre de la variation entre plis). `class_weight="balanced"` déplace surtout le seuil : il augmente fortement le rappel à 0,5, mais pas le F2 maximal.

7.3 Optimisation, calibration et seuils

Optimisation. `RandomizedSearchCV`, 16 combinaisons, sur un espace qui couvre les deux familles (l'étape `model` est elle-même un hyperparamètre), avec et sans pondération. Meilleure combinaison : HistGradientBoosting balanced, `learning_rate=0.03`, `max_iter=300`, `max_leaf_nodes=127`, `min_samples_leaf=20`,

`l2_regularization=1.0` (F2 maximal 0,7918, contre 0,7905 pour la meilleure forêt). Il est aussi plus rapide (8 s contre 11 à 16 s) et plus léger.

Calibration. La pondération gonfle les probabilités. L'étape `model` est donc enveloppée dans `CalibratedClassifierCV` (isotonique, 3 plis) : score de Brier de 0,131 à 0,115 sur le test (0,200 sans modèle).

Seuils de décision, choisis sur les probabilités hors pli de l'entraînement :

- seuil F2 = 0,171 : maximise le F2 ; intégré au modèle par `FixedThresholdClassifier`, donc appliqué par `predict()` ;
- seuil F0,5 = 0,537 : maximise le F0,5 (précision prioritaire) ; utilisé pour les actions contraignantes.

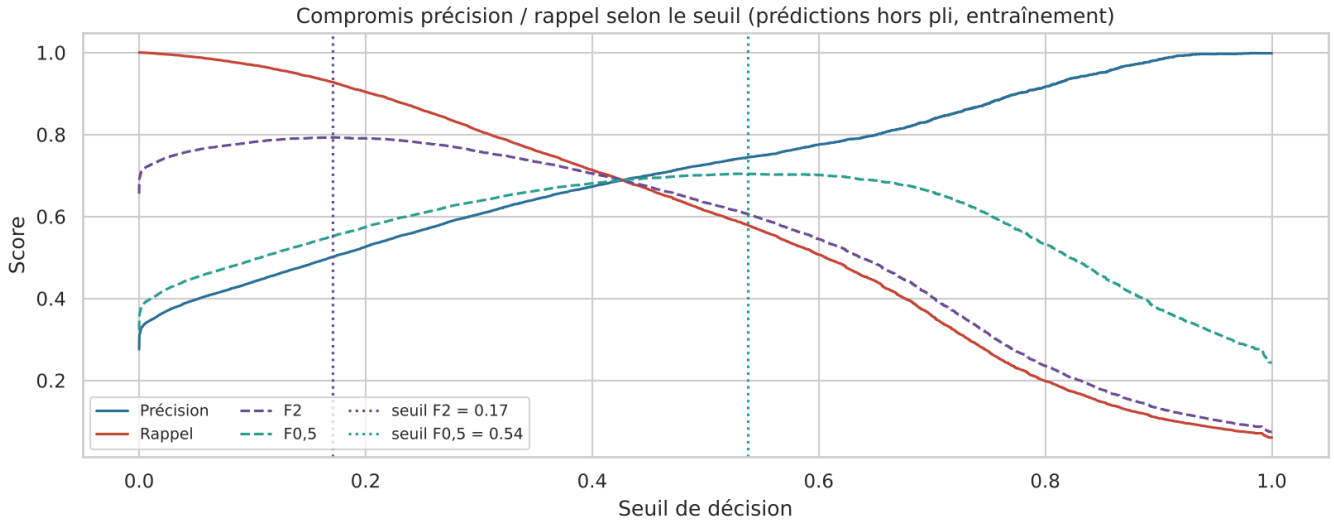


Figure 2. Précision, rappel, F2 et F0,5 selon le seuil (prédictions hors pli, entraînement)

7.4 Résultats sur le test

Seuil	Exactitude	Précision	Rappel	F1	F2
F2 (0,17)	**73,2 %**	**50,8 %**	**93,1 %**	**0,657**	**0,798**
F0,5 (0,54)	83,2 %	74,9 %	58,7 %	0,658	0,613
0,50	83,2 %	73,3 %	61,7 %	0,670	0,637

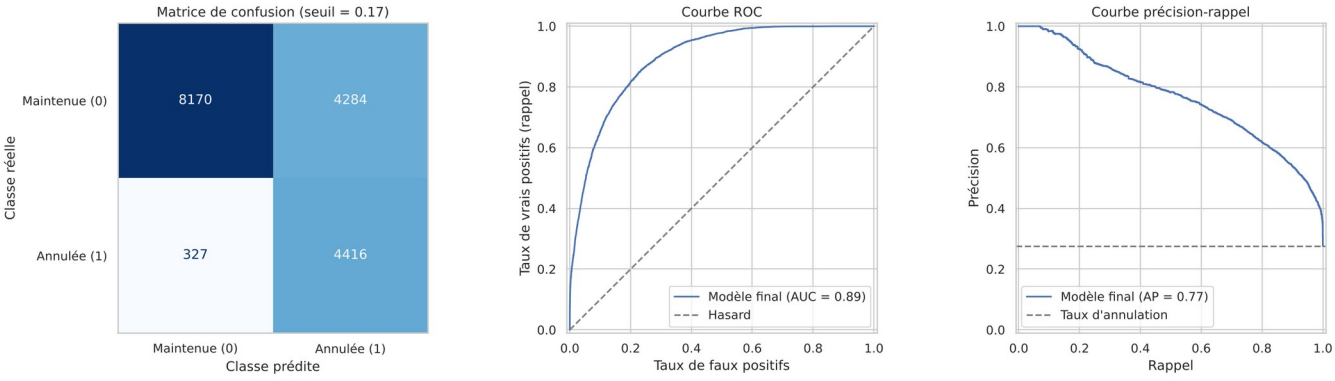


Figure 3. Matrice de confusion au seuil F2, courbe ROC et courbe précision-rappel (test)

Le F2 maximal du test (0,798) rejoint celui de la validation croisée (0,792) : la performance annoncée est fiable. L'écart de ROC-AUC entre entraînement (0,931) et test (0,895) indique un léger surapprentissage sans conséquence.

Variables déterminantes (importance par permutation, baisse de PR-AUC) : `lead_time` (0,143), `country` (0,138), `market_segment` (0,114), `total_of_special_requests` (0,104), `adr` (0,073), `required_car_parking_spaces` (0,072), `previous_cancellations` (0,063).

SECTION 08

Analyse prescriptive

En bref Les annulations sans dépôt coûtent 3,0 M€ net sur la période (11,7 % du revenu réalisé), dont 53 % viennent des annulations du dernier mois. Un acompte réservé aux réservations à risque élevé en couvrirait l'essentiel.

Principe : aucun chiffre inventé. Délais, taux de revente et pertes sont calculés à partir des données (`src/prescriptive.py`). Seuls le niveau d'acompte et le risque de relogement accepté sont des choix de l'hôtel, présentés comme des scénarios. L'analyse utilise `reservation_status_date`, exclue du modèle : c'est légitime pour un bilan après coup.

Données utilisées. Le bilan financier garde les doublons (119 208 réservations), car deux lignes identiques sont souvent deux vraies chambres d'un même groupe. Les scénarios qui utilisent le modèle portent sur l'ensemble de test sans doublons.

8.1 Taux de revente : méthode

1. Pour chaque nuit et chaque hôtel, compter les chambres réellement occupées.
2. Estimer la capacité par le 99e centile de l'occupation : 224 chambres (City Hotel), 184 (Resort Hotel).
3. Chaque nuit, les chambres annulées sont perdues dans la limite des chambres restées vides ; le reste est considéré comme revendu.
4. Moyenne par annulation, puis par tranche de délai, pondérée par le revenu du séjour.

Formules

```
perdues (nuit) = min(annulées, capacité - occupées)
perte nette    = revenu × (1 - taux de revente)
```

8.2 Moment des annulations, revente et pertes

Annulation faite… avant l'arrivée	Annulations	Part	Part de groupes	Taux de revente	Perte nette	Part de la perte
moins d'une semaine	5 411	18,2 %	11,6 %	61,0 %	663 948 €	21,8 %
1 semaine à 1 mois	8 257	27,8 %	11,2 %	71,1 %	965 127 €	31,7 %
1 à 2 mois	5 337	18,0 %	8,2 %	77,1 %	541 508 €	17,8 %
2 à 3 mois	3 084	10,4 %	10,6 %	80,6 %	300 540 €	9,9 %
plus de 3 mois	7 580	25,5 %	8,7 %	86,2 %	571 563 €	18,8 %

Manque à gagner brut : 13,1 M€ ; net après revente : 3,0 M€ ; revenu réalisé : 26,0 M€. Exclure le premier et le dernier mois (incomplets) change les taux de revente de moins d'un point.

8.3 Test d'hypothèse sur l'acompte

Le chi-deux entre `deposit_type` et `is_canceled` est très significatif (2 315, p < 0,001), mais dans le mauvais sens : 95 % des réservations non remboursables sont annulées. Ce sont surtout des blocs d'agences (958 sur 1 010 portugaises, 278 du seul agent n° 1), intégralement payés. À segment égal, les effectifs sont trop petits (18 en agence en ligne). Les données ne mesurent ni l'effet dissuasif d'un acompte, ni le nombre de clients fidèles qu'il ferait fuir : il faut un test A/B (de 109 à 832 réservations par groupe selon l'effet recherché, risque 5 %, puissance 80 %).

8.4 Scénarios d'acompte

Ensemble de test, acompte demandé au-dessus du seuil F0,5 : 3 713 réservations, dont 2 782 annulations et 931 clients fidèles ; perte nette sans acompte : 318 018 €.

Politique	Acompte conservé	Perte nette couverte	Taux de fuite maximal acceptable
20 % du séjour	294 430 €	93 %	62 %
Première nuit	334 706 €	105 %	70 %
50 %, remboursable jusqu'à 30 jours	229 206 €	72 %	48 %
100 %, remboursable jusqu'à 30 jours	**458 413 €**	**144 %**	**96 %**
50 % du séjour	736 074 €	232 %	154 %

Taux de fuite maximal acceptable = acompte conservé / revenu des fidèles exposés : au-delà, l'acompte coûte plus qu'il ne rapporte. L'acompte est conservé même si la chambre est revendue, d'où des couvertures supérieures à 100 %. Au seuil F2, 4 284 fidèles seraient exposés et ce taux tomberait entre 21 % et 56 %.

8.5 Action par réservation et surréservation

Zone	Probabilité	Action	Réservations	Prévu / observé
Faible	< 0,17	aucune	8 497	4,4 % / 3,8 %
Modéré	0,17 à 0,54	relance sans engagement	4 987	31,8 % / 32,8 %
Élevé	≥ 0,54	acompte et surréservation	3 713	74,1 % / 74,9 %

Surréservation. Le nombre de présents suit une loi de Poisson binomiale (chaque réservation a sa probabilité), calculée exactement par convolution. On retient le plus grand nombre de réservations supplémentaires tel que le risque de reloger au moins un client reste sous 5 %. Exemple du 28 avril 2017, City Hotel (34 réservations au carnet) : 11,8 annulations prévues, 13 observées ; 11 réservations de plus font passer l'occupation espérée de 65 % à 86 %, avec 3,4 % de risque de relogement.

SECTION 09

Application Streamlit

En bref `streamlit run app/app.py` lance une application en 4 onglets qui recharge le pipeline entraîné et la logique de `src/prescriptive.py`, sans aucun calcul dupliqué.

Onglet	Entrées	Sorties
Réservation	formulaire d'une réservation brute (27 champs)	jauge de probabilité avec les seuils F2 et F0,5, zone de risque, action recommandée, revenu du séjour, perte espérée, sensibilité au délai
Lot de réservations	échantillon du test ou fichier CSV importé	annulations attendues, répartition par zone, liste triée par perte espérée, export CSV
Pertes et acomptes	type d'acompte (part ou nuits), montant, délai de remboursement, seuil (F0,5, F2 ou libre)	manque à gagner brut et net par tranche ; acompte conservé, perte restante, fidèles exposés, taux de fuite maximal acceptable
Surréservation	hôtel, date, capacité, risque de relogement accepté	réservations supplémentaires, occupation espérée, risque, annulations prévues et observées

Fonctionnement. Le modèle et les calculs lourds (occupation nuit par nuit) sont mis en cache (`st.cache_resource`, `st.cache_data`) et calculés une fois par session. Au chargement, `load_pipeline()` rétablit la sortie pandas entre les étapes du pipeline.

SECTION 10

Reproductibilité

En bref Le projet se réinstalle et se réexécute à l'identique : versions fixées, graine aléatoire unique (`RANDOM_STATE = 42`), données téléchargées automatiquement, modèle entraîné inclus dans le dépôt.

Installation (Python 3.11 minimum, testé avec Python 3.13)

```
git clone https://github.com/lezoul/hotel-cancellation-pipeline.git
cd hotel-cancellation-pipeline
python -m venv .venv
.venv\Scripts\Activate.ps1        # macOS / Linux : source .venv/bin/activate
pip install -r requirements.txt
```

Étape	Commande	Durée
Application (directement)	`streamlit run app/app.py`	

Étape	Commande	Durée
Pipeline et modèle	exécuter `notebooks/01_pipeline.ipynb`	environ 20 min
Analyse prescriptive	exécuter `notebooks/02_analyse_prescriptive.ipynb`	environ 2 min
Tests	`python -m pytest -q`	quelques secondes

Versions principales (`requirements.txt`) : pandas 3.0.5, numpy 2.5.3, scikit-learn 1.9.1, scipy 1.18.1, matplotlib 3.11.2, seaborn 0.13.2, plotly 7.1.0, streamlit 1.66.1.

Tests unitaires (12 tests) : variables créées et noms de sortie, entrée non modifiée, modalités apprises sur `fit` seulement, bornes d'écrêtage, transformateurs clonables, pipeline sans valeur manquante ni perte de lignes, tranches de délai, loi de Poisson binomiale, surréservation, remboursement des acomptes.

Qualité du code. Code commenté ligne par ligne ; vérification de types (pyright avec les définitions de types de pandas, comme Pylance dans VS Code) sans erreur sur `src/`, `app/`, `tests/` et les deux notebooks. Dépôt Git avec un commit par étape ; le pipeline entraîné (`models/`, environ 9 Mo) est versionné.

SECTION 11

Limites et pistes d'amélioration

En bref Le modèle est fiable sur ces deux hôtels et cette période ; une mise en production demande quelques vérifications.

Limite	Conséquence	Piste
Deux hôtels portugais, 2015 à 2017	le modèle ne se transpose pas tel quel	réentraîner sur les données de l'établissement visé
Séparation aléatoire	performance peut-être optimiste face à l'avenir	validation temporelle : entraîner sur 2015 et 2016, tester sur 2017
`required_car_parking_spaces` (0 % d'annulation dès qu'elle est positive) et `country` (part anormale du Portugal)	fuite partielle possible si ces champs sont remplis à l'arrivée	vérifier auprès de l'hôtel quand ils sont saisis
Capacité estimée, même taux de revente pour toutes les annulations d'une nuit	taux de revente approximatifs	utiliser la capacité réelle et l'historique des relocations
Effet réel d'un acompte inconnu	les scénarios donnent un seuil de rentabilité, pas un gain garanti	test A/B sur des réservations à risque élevé tirées au hasard
Doublons supprimés pour le modèle	de vraies chambres de groupes peuvent avoir été retirées	identifiant de réservation dans une future version des données

Sources

- Antonio, N., de Almeida, A. et Nunes, L. (2019). Hotel booking demand datasets. Data in Brief, 22, 41-49.
- Fiche TidyTuesday du jeu de données (2020-02-11) : https://github.com/rfordatascience/tidytuesday/blob/main/data/2020/2020-02-11/readme.md
- Cours « Pipelines de prétraitement », IFM30513, automne 2026, Prof. Nadjate Saïdani.